Morpho: Public Allocator Security Review

Auditors

MiloTruck, Lead Security Researcher

Saw-mon and Natalie, Lead Security Researcher

Om Parikh, Security Researcher

Report prepared by: Lucas Goiriz

August 19, 2026

Contents

1 About Spearbit

Spearbit is a decentralized network of expert security engineers offering reviews and other security related services to Web3 projects with the goal of creating a stronger ecosystem. Our network has experience on every part of the blockchain technology stack, including but not limited to protocol design, smart contracts and the Solidity compiler. Spearbit brings in untapped security talent by enabling expert freelance auditors seeking flexibility to work on interesting projects together.

Learn more about us at spearbit.com

2 Introduction

Morpho is a trustless and efficient lending primitive with permissionless market creation.

Disclaimer: This security review does not guarantee against a hack. It is a snapshot in time of Morpho: Public Allocator according to the specific commit. Any modifications to the code will require a new security review.

3 Risk classification

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

3.1 Impact

- High - leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium - global losses <10% or losses to only a subset of users, but still unacceptable.
- Low - losses will be annoying but bearable--applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

3.2 Likelihood

- High - almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium - only conditionally possible or incentivized, but still relatively likely
- Low - requires stars to align, or little-to-no incentive

3.3 Action required for severity levels

- Critical - Must fix as soon as possible (if already deployed)
- High - Must fix (before deployment if not already deployed)
- Medium - Should fix
- Low - Could fix

4 Executive Summary

Over the course of 2 days in total, Morpho engaged with Spearbit to review the vault-v2 protocol. In this period of time a total of **17** issues were found.

Summary

Project Name	Morpho
Repository	vault-v2
Commit	10f16971
Type of Project	DeFi, Lending
Audit Timeline	Jul 29th to Jul 31st

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	6	2	4
Gas Optimizations	0	0	0
Informational	11	4	7
Total	**17**	**6**	**11**

The Spearbit team reviewed Morpho's vault-v2 holistically on commit hash 2b139002 (release 2026-08-13) and concluded that all findings were addressed and no new vulnerabilities were identified.

4.1 Scope

The security review had the following components in scope for vault-v2 on commit hash 10f16971:

```
src/periphery/blue-public-allocator
├── BluePublicAllocator.sol
└── interfaces
    └── IBluePublicAllocator.sol
```

5 Findings

5.1 Low Risk

5.1.1 Reallocation lacks a vault-wide kill switch

Severity: Low Risk.

Context: BluePublicAllocator.sol#L98-L99.

Description: `allocateFromIdle` has one switch for each vault: `vaultData[v].canAllocateFromIdle`. An allocator can turn off this switch to stop all public allocations from idle.

`reallocate` has no similar switch. Let D_v be the markets where `canDeallocate[vault][id]` is true. A route stays enabled when its source is in D_v, both adapters are active, and its destination is below its cap. To stop all public reallocations, an allocator must disable every source market or every active adapter:

$$\text{disableReallocation}(v) \implies D_v := \quad \text{or} \quad \text{ActiveAdapters}_v := .$$

This requires a complete list of the prior settings and may take multiple transactions. If the allocator misses one market-adapter route, anyone can still call `reallocate` through it.

During an incident, a vault cannot stop all public reallocations with one state change. It must also change and later restore its market or adapter settings.

Recommendation: Add `canReallocate` to `VaultData`. Add an allocator-only setter and event. Check the flag at the start of `reallocate`, before native penalty accounting. Keep the existing market and adapter checks. The allocator can then stop or resume all public reallocations with one change for that vault.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.1.2 Public callers can shift pending Morpho bad debt to the vault

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Description: `BluePublicAllocator` lets any caller move vault assets into an enabled Morpho market. `reallocate` takes the assets from an enabled source market. `allocateFromIdle` takes them directly from the vault's idle balance. The destination caps bound the allocation size, but they do not restrict its timing.

All monetary variables below are amounts of Market B's loan token. They are not numbers of Morpho supply shares. Positions are valued at the pre-loss supply-share exchange rate.

Symbol	Unit	Meaning
S	loan-token assets	Market B's `totalSupplyAssets` before the attack.
U	loan-token assets	Market B's `totalBorrowAssets` before the attack.
$L = S - U$	loan-token assets	Market B's available accounting liquidity.
D	loan-token assets	`badDebtAssets` that the next liquidation will realize.
V	loan-token assets	The vault's current Market B supplier claim.
R	loan-token assets	The adversary's current Market B supplier claim.
X	loan-token assets	Vault assets that the adversary allocates to Market B.

Y	loan-token assets	Assets that the adversary withdraws after the allocation and before liquidation.

Assume $0 \leq D \leq U \leq S$, $0 \leq V + R \leq S$, and $X \geq 0$. Assume Market B has at least X loan-token assets of public cap headroom. Also assume X is available from the vault's idle balance or an enabled source market.

At T_1, the adversary calls `allocateFromIdle` or `reallocate` and supplies X vault assets to Market B. Morpho mints supply shares before it records D. The vault's new assets therefore participate in the pending loss.

The new supply also adds X liquidity. At T_2, the adversary can withdraw:

$$0 \leq Y \leq \min(R, L + X).$$

The adversary can fully exit when:

$$X \geq \max(0, R - L).$$

At T_3, the liquidation subtracts D from `totalSupplyAssets` without burning supply shares. Assume that no interest or other balance change occurs between T_1 and T_3. Ignoring virtual shares and rounding, supply and withdrawal preserve the pre-loss exchange rate. The vault's loss changes from:

$$D\frac{V}{S}$$

to:

$$D\frac{V + X}{S + X - Y}.$$

Both fractions are dimensionless. Both loss expressions therefore have loan-token asset units.

With $Y = 0$, the new vault shares dilute the loss of existing suppliers. With $Y = R$, the adversary first replaces its risky claim with cash. The vault and the other suppliers then absorb the fixed bad debt. The attack does not create more bad debt; it changes who bears D.

The public and Vault V2 caps bound X and the vault's final loss. However, they also authorize an arbitrary caller to fill that headroom immediately before a known loss. The fixed native penalty does not scale with X or the loss that the adversary avoids.

`allocateFromIdle` also consumes assets that were immediately available for vault withdrawals. In both paths, `VaultV2.allocate` accrues fees at the pre-loss value. This can crystallize performance and management fees before the write-down. After liquidation, the cached market allocation can remain above its real value until another adapter interaction updates it.

Proof of Concept: Assume the following state. All amounts are Market B loan-token units. Ignore virtual shares and rounding, and normalize the pre-loss supply-share exchange rate to one:

- Market B has S = 100 supply assets and U = 100 borrow assets;
- Available liquidity is L = 0;
- The adversary's current supplier claim is R = 20;
- The vault has no prior Market B position and has 20 idle loan-token assets;
- The next liquidation realizes D = 40 bad debt;
- Market B is active, `canAllocateFromIdle` is true, and its caps have 20 loan-token units of headroom.

At T_1, the adversary calls `allocateFromIdle` for X = 20. Market B now has 120 supply assets, 100 borrow assets, and 20 available liquidity.

At T_2, the adversary withdraws $Y = R$ = 20 assets. Market B returns to 100 supply assets and 100 borrow assets. The vault now holds the shares that replaced the adversary's position.

At T_3, the liquidation realizes 40 bad debt. The vault owns about 20% of the remaining supply shares and loses about 8 loan-token units. Without the sequence, the adversary would have lost about 8 loan-token units. The adversary instead exits at the pre-loss value and pays only gas and the configured native penalty.

The same construction uses `reallocate` when an enabled source market has 20 withdrawable loan-token assets.

Recommendation: Separate risk caps from public allocation authority. Add a destination-specific permission for public inflows and default it to false. Give `allocateFromIdle` its own destination permissions. If public reallocation remains enabled, limit it to explicit source-destination routes or designated sink markets.

More fine-grained permissions would be the following edge-flow permissions:

`idle` $\rightarrow (A, M_{A,i})$ $(B, M_{B,j}) \rightarrow (A, M_{A,i})$.

Treat every enabled public destination cap as an amount that an adversary can fill before a known loss. Set its public cap to the current allocation, or to zero, when forced exposure is not acceptable. Keep `canAllocateFromIdle` disabled unless every permitted destination can safely receive an adversarially timed allocation. Minimum-idle limits, inflow rate limits, and amount-based penalties can further bound or deter the attack.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.1.3 Public callers can manipulate Morpho rates with vault liquidity

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Description: `BluePublicAllocator` lets any caller choose the source, destination, amount, and time of an enabled allocation. The checks only enforce active adapters, source permission, destination caps, Vault V2 caps, and the fixed native penalty. They do not constrain the route's effect on utilization or rates.

Let a Morpho market have S supply assets and B borrow assets. Its utilization is:

$$U = \frac{B}{S}.$$

If a caller moves X vault assets out of a source market A, then:

$$U'_A = \frac{B_A}{S_A - X}, \qquad 0 \leq X \leq S_A - B_A.$$

The caller can therefore raise U'_A to one. A supplier in A receives a higher supply rate, while borrowers in A pay a higher borrow rate. The higher debt growth can also make marginal borrowers unhealthy sooner.

If the caller moves the same assets into a destination market B, then:

$$U'_B = \frac{B_B}{S_B + X}.$$

A borrower in B receives a lower borrow rate. The lower debt growth can delay an interest-driven liquidation. The borrower may instead borrow the new liquidity, subject to Morpho's health check. This increases utilization again and reduces the rate benefit, but gives the borrower access to vault-funded liquidity.

A caller can leave A above target utilization to raise its target rate. The same caller can leave B below target utilization to lower its target rate. Each market records its own change when it next accrues interest. Morpho accrues interest before each supply or withdrawal, so the manipulation does not change interest from an earlier interval.

An attacker can hold a supplier position in A and a borrow position in B. One `reallocate` then improves both positions. The attacker pays one fixed native penalty, while the benefit scales with X, both positions, and the duration of the rate change. `allocateFromIdle` provides the borrower-side effect without an enabled source market.

The public and Vault V2 caps bound the allocation at one time. They do not directly constrain rate movement, enforce a minimum source reserve, compare source and destination yield, or bind permissions to a specific source-destination route. Short-term vault deposits can also increase relative-cap headroom before a later public allocation. This requires temporary capital. The implementation expressly identifies this relative-cap manipulation.

Proof of Concept: Assume both markets use the initial Adaptive Curve target rate of 4% per year. Ignore fees, interest during the reallocation, and rounding. Let the attacker hold a supplier position in Market A and a borrow position in Market B.

- Market A has S_A = 100 and B_A = 90. The vault supplies X = 10 of Market A's assets.
- Market B has S_B = 10 and B_B = 10. The attacker holds Market B's full borrow position.
- Both adapters are active, Market A can be deallocated, Market B has at least 10 units of public and Vault V2 cap headroom, and the caller pays the configured native penalty.

Before the reallocation,

$$U_A = 90\%, \qquad U_B = 100\%.$$

At T_1, the attacker calls `reallocate` for X = 10 from Market A to Market B. The new utilizations are:

$$U'_A = \frac{90}{100 - 10} = 100\%, \qquad U'_B = \frac{10}{10 + 10} = 50\%.$$

At a fixed target rate, the Adaptive Curve gives these approximate annualized rates:

Position	Before	After
Market A gross supplier rate	3.6%	16%
Market A borrow rate	4%	16%
Market B borrow rate	16%	2.67%
Market B gross supplier rate	16%	1.33%

The attacker receives the higher Market A supplier rate and the lower Market B borrow rate. The vault moves from the approximately 3.6% Market A supply rate to the approximately 1.33% Market B supply rate.

At $T_2 = T_1 + 1$ day, an interaction in each market can record its IRM adaptation. The configured speed is 50 per year. Market A's target rate is multiplied by approximately 1.147, while Market B's target rate is multiplied by approximately 0.941. Each new value becomes the starting target rate for later adaptation, even if utilization changes.

Recommendation: Separate market risk caps from public route authority. Configure explicit source-destination routes instead of composing every enabled source with every enabled destination.

Fine-grained edge-flow permission system would be indexed by:

- `idle` $\rightarrow (A, M_{A,i})$

- $$(B, M_{B,j}) \to (A, M_{A,i})$$

Bound each route's economic effect. Enforce a minimum source reserve and maximum utilization change. Limit destination inflows over time, and reject a route when the destination supply rate is below an accepted threshold relative to the source. Keep `allocateFromIdle` disabled when arbitrary borrowers must not direct idle assets into their markets.

Use an amount-based penalty so the cost scales with the allocation. Add a cooldown or rate limit to bound repeated changes. Do not rely on relative caps after short-term deposits; configure the public absolute cap as the binding limit.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.1.4 Public allocation can strip Vault withdrawal liquidity

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Description:

- `BluePublicAllocator`'s `reallocate` lets any caller paying the flat native penalty move a *caller-selected amount* from any publicly deallocatable market to any active market within its caps.
- `allocateFromIdle` similarly lets a public caller move idle assets when `canAllocateFromIdle` is enabled.

Neither path preserves a source-side liquidity floor, rate-limits outflow, protects the Vault's configured liquidity route, nor requires the destination to expose a permissionless return path.

This conflicts with the withdrawal path: `VaultV2.exit` uses the Vault's idle balance and then deallocates only from `liquidityAdapter` with `liquidityData`; it does not search arbitrary adapters or markets. An adversary can therefore route the idle buffer or the designated liquidity-market position into a non-liquidity market. A sufficiently collateralized borrower can consume the destination liquidity, after which Morpho refuses deallocation that would leave borrow assets above supply assets.

Consequently, ordinary supplier withdrawals can revert. Recovery requires a trusted allocator, another caller to pay the public-allocation penalty for a reverse move (only if the destination is publicly deallocatable and liquid), or `forceDeallocate`. The latter burns supplier shares worth approximately the configured penalty times the forced amount, up to 2%; it can also fail while the destination is borrowed out. The action can be repeated whenever deposits or allocator intervention replenish the protected source.

The issue is conditional on explicit curator configuration: `BluePublicAllocator` must be authorized, the relevant public-allocation flags and caps must be enabled, and the flat penalty must be economically acceptable. Caps and the penalty bound/deter the action, while `forceDeallocate` prevents a permanent lock. They do not, however, bound the withdrawal-liquidity externality by the caller's cost. The adversary need not steal Vault assets; obtaining preferred borrow liquidity, suppressing competing withdrawals, or griefing suppliers can provide the incentive.

Proof of Concept: Let I be idle assets, L the assets in the configured liquidity market, f the flat native penalty, and p the destination adapter's force-deallocation penalty.

1. Configure `BluePublicAllocator` as an allocator, mark the exact liquidity market publicly deallocatable, and cap a distinct destination market for at least L.
2. The adversary pays f and calls `reallocate(vault, liquidityAdapter, liquidityData, otherAdapter, otherMarket, L)`.
3. A healthy Morpho account borrows the liquidity in `otherMarket`.

4. A supplier requests a withdrawal greater than I. `VaultV2.exit` tries only `liquidityAdapter/liquidityData`; because that position was drained, the withdrawal reverts.
5. Moving the assets back costs another f and succeeds only when the destination is publicly deallocatable and has available liquidity. Otherwise the supplier must wait or force-deallocate, losing shares worth approximately pD for forced amount D. Replenishing L permits the cycle to repeat.

The idle construction is analogous: enable `canAllocateFromIdle`, then call `allocateFromIdle(vault, otherAdapter, otherMarket, I)`. A withdrawal fails whenever the remaining configured liquidity-route capacity is insufficient. `forceDeallocate` and `withdraw` can be batched with `multicall` to remove the inter-transaction race, but this does not eliminate the penalty or make a borrowed-out destination liquid.

Recommendation: Preserve an explicit withdrawal-liquidity reserve on both public paths. In particular, prevent public deallocation from the current `liquidityAdapter/liquidityData` below a curator-set floor and prevent `allocateFromIdle` from reducing idle assets below a separate floor. Add per-epoch source outflow limits or cooldowns so a flat fee cannot move an unbounded balance.

Additionally require every public destination to expose a symmetric permissionless path back to the configured liquidity route, with compatible caps and preferably no second fee for restoring the reserve. Consider pricing public allocation proportionally to the amount or externality rather than solely per call. Add regression tests covering a full source drain, destination borrowing, ordinary-withdrawal failure, unavailable reversal, penalized recovery, and a repeated replenishment cycle.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.1.5 `BluePublicAllocator` accepts vaults and adapters not deployed by their factories

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Description: Let F_V be the relevant `VaultV2Factory` and F_A the relevant `MorphoMarketV1AdapterV2Factory`. The documented configuration domain is:

$$V \in \text{Vaults}(F_V), \qquad A \in \text{Adapters}(F_A), \qquad \text{parentVault}(A) = V.$$

Both factories expose membership mappings for these checks. However, every setter that accepts `vault` authenticates the caller only through `IVaultV2(vault).isAllocator(msg.sender)`. It does not first require `VaultV2Factory.isVaultV2(vault)`. Since `vault` is caller-supplied, any contract can implement this selector and choose which callers pass.

`setIsActiveAdapter`, `setAbsoluteCap`, and `setCanDeallocate` also do not require `MorphoMarketV1AdapterV2Factory.isMorphoMarketV1AdapterV2(adapter)` or verify that the adapter belongs to `vault`. They store configuration based only on the supplied addresses.

Consequently, `BluePublicAllocator` stores state and emits configuration events for unsupported vaults. For a legitimate vault, an authorized allocator can also enable an unsupported adapter. The public allocation paths then rely on an unenforced assumption: the adapter must use the same per-market identifier and allocation semantics as `MorphoMarketV1AdapterV2`. A different adapter can make the public cap check inspect an unrelated identifier, or it can make public allocation calls revert.

An arbitrary caller cannot configure a factory-deployed vault unless that caller is its allocator. The vault's curator must also have added an unsupported adapter and configured its Vault V2 caps before public calls can route assets through it. The issue therefore requires prior curator configuration and an allocator mistake or compromise. Separately, an off-chain consumer can be affected if it treats unverified state or events as canonical.

Recommendation: Store the relevant vault and adapter factories as immutable configuration. Before each setter updates state, require that the vault is registered by the vault factory. For each adapter-setting function, also require that the adapter is registered by the adapter factory and that its `parentVault` equals `vault`. If several factories are supported, validate them through an allowlisted factory registry.

Add tests with a contract that spoofs `isAllocator`, a directly deployed adapter, and a factory-deployed adapter whose parent is a different vault.

Morpho: Fixed in PR 964.

Spearbit: PR 964 adds the `VaultV2Factory` membership check to `BluePublicAllocator`'s setter functions. The adapter factory check is intentionally omitted to preserve compatibility with future adapters that do not diverge significantly from `MorphoMarketV1AdapterV2`'s behavior.

5.1.6 `nativePenalty` could be cheaper than the `forceDeallocate` penalty

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Description: In `BluePublicAllocator`, the cost of reallocating liquidity from one adapter to another via `reallocate()` is a flat fee:

```
require(msg.value == vaultData[vault].nativePenalty, IncorrectNativePenalty());
```

Whereas in `VaultV2`, the fee in `forceDeallocate()` is a percentage of the assets deallocated:

```
uint256 penaltyAssets = assets.mulDivUp(forceDeallocatePenalty[adapter], WAD);
```

Considering that `nativePenalty` is a flat fee while `forceDeallocatePenalty` is a percentage fee, if:

- `VaultV2` has a `liquidityAdapter`.
- `liquidityAdapter` is also configured as an active adapter in `BluePublicAllocator`.

For a sufficiently large asset amount, it will cost less fees to reallocate into `liquidityAdapter` followed by withdraw, instead of using `forceDeallocate()`. This could be seen as a way to ”bypass” the force-deallocation fee configured in `VaultV2`.

Recommendation: A possible fix would be to change reallocation fee to be based on `assets`. Alternatively, simply document this case so that allocators either avoid such a configuration, or choose to configure a larger `nativePenalty`.

Morpho: Fixed in PR 966.

Spearbit: PR 966 replaces the flat native penalty with a relative fee paid in the vault's loan asset, so curators can configure the penalty independently to be lower or higher than the `forceDeallocate` penalty.

5.2 Informational

5.2.1 Native-currency penalties cannot be enforced on Tempo

Severity: Informational.

Context: IBluePublicAllocator.sol#L9.

Description: Let p_V be a vault's (V) `nativePenalty`. Both public allocation paths require `msg.value == p_V` and accrue `msg.value`; claims likewise use a native-value `CALL`.

Tempo has no native token: `CALLVALUE` always returns zero and nonzero native-value transactions are rejected. Hence.

$$p_V > 0 \implies \texttt{reallocate} \text{ and } \texttt{allocateFromIdle} \text{ are unreachable,}$$
$$p_V = 0 \implies \text{both calls remain unpenalized.}$$

Thus no configuration enforces a positive penalty on Tempo. Setting one instead disables public allocation for the vault.

Recommendation: Make the penalty asset/token configurable per vault and collect it with `safeTransferFrom`; use a TIP-20 token on Tempo. Claim accrued penalties with the corresponding token transfer. If native penalties are retained as an optional mode, reject nonzero native configuration on Tempo.

Morpho: Fixed in PR 966.

Spearbit: PR 966 resolves this by replacing the native-currency penalty with a relative fee collected in the vault's underlying loan token via `safeTransferFrom`, which functions correctly on chains like Tempo that reject nonzero native-value transactions.

5.2.2 Any allocator can divert accrued native penalties

Severity: Informational.

Context: BluePublicAllocator.sol#L87-L94.

Description: Let P_V be the native penalty accrued by public allocation calls for vault V, and let A_V be its set of allocators. `claimNativePenalty` implements.

$$\forall a \in A_V,\ \forall r: \qquad \text{claim}(a, V, r) \implies \text{balance}(r) \mathrel{+}= P_V, \quad P_V := 0.$$

Thus the first allocator to claim chooses the beneficiary. A malicious or compromised allocator can send the entire accrued amount for that vault V to itself, although the allocator role otherwise routes vault assets within curated-defined limits. The penalty paid by users of the public allocator consequently benefits an arbitrary allocator-selected account rather than the vault's lenders.

This differs from `VaultV2.forceDeallocate`: the penalty withdrawal uses `address(this)` as receiver. The assets remain idle in the vault and can be reallocated; because both recorded assets and share supply decrease while real assets remain, the surplus is subsequently recognized as interest, subject to `maxRate`, for the remaining lenders.

Recommendation: Remove the caller-supplied receiver. If the penalty is intended for lenders, collect it in `vault.asset`, or convert (or wrap) it under curation-defined slippage bounds, and transfer the resulting assets to the vault. Otherwise, store a beneficiary per vault and permit only a timelocked curation action to change it.

Morpho: PR 966 partially mitigates this: the new loan-token relative penalty is donated directly to the vault (analogous to `forceDeallocate`), removing the separate claim step and the allocator's ability to divert accrued penalty funds.

Spearbit: Fix verified.

5.2.3 Fixed factory salt prevents per-vault adapter policy segmentation

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: Let F be a `MorphoMarketV1AdapterV2Factory` with immutable Morpho and IRM addresses M and I, and let V be a parent vault. The factory deploys with the constant salt 0, so every call uses the same CREATE2 address.

$$A = \text{CREATE2Address}\left(F, 0, \text{keccak256}(\texttt{creationCode} \parallel \text{encode}(V, M, I))\right).$$

The first call deploys A. Every subsequent call for V attempts to deploy at the occupied address A and reverts. Hence a single factory satisfies.

$$|\text{Adapters}_F(V, M, I)| \leq 1.$$

This prevents two adapter-scoped policies over markets sharing the same parent vault, Morpho instance, and IRM. In particular, every market routed through A returns the same `adapterId`, so the adapter-wide cap can define only one aggregate basket. Likewise, `forceDeallocatePenalty[A]` applies to every such market. The vault therefore cannot, through one factory, partition those markets into independently capped baskets or assign different force-deallocation penalties to them.

Deploying another factory or bypassing the canonical factory produces a distinct adapter address, but may be incompatible with a registry intended to recognize adapters from the configured factory. The limitation is therefore structural whenever deployment or registry policy admits only that factory.

Recommendation: Permit multiple deployments per parent vault by accepting a caller-supplied salt or maintaining a per-vault deployment nonce. Track the resulting adapters as a set or list while retaining `isMorphoMarketV1AdapterV2` for registry membership. If one adapter per factory and vault is intentional, document that separate factory deployments are required for adapter-scoped cap baskets and force-deallocation penalties, and ensure the registry can authorize all such factories.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.2.4 `BluePublicAllocator` does not enforce the specs provided in the docs

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: According to Morpho Public Allocator Docs:

> Vault curators control how the Public Allocator can move their assets by setting the Public Allocator contract with the Allocator role, and then sets:
> - **Flow Caps**: Maximum amounts that can flow in or out of specific markets.
> - **Market Preferences**: Which markets can send or receive assets.
> - **Fee Parameters**: Optional fees that curators can set and collect for reallocations (these fees go to the **curator**, not to Morpho).

The current implementation does not:

- Restrict the maximum amount flowing out of markets.

For a more detailed analysis refer to:

- Public callers can shift pending Morpho bad debt to the vault.
- Public callers can manipulate Morpho rates with vault liquidity.

Recommendation: The implementation or the docs would need to be updated to conform to each other.

Morpho: Acknowledged. The referenced documentation describes the Vault V1.x Public Allocator, which has a different design; the docs will be updated to highlight this distinction after deployment.

Spearbit: Acknowledged.

5.2.5 BluePublicAllocator use cases and economic boundaries

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description:

Purpose: `BluePublicAllocator` gives the public limited use of a Vault V2 allocator permission. A public caller can move the vault asset through one of two paths:

1. `allocateFromIdle` moves assets from the vault's idle balance to one Morpho market.
2. `reallocate` withdraws assets from one Morpho market and supplies the same nominal amount to another Morpho market.

The caller never receives the moved assets from `BluePublicAllocator`. The caller must obtain value from another action or position. This fact determines which use cases have a direct economic incentive.

The main use case is borrower-triggered, just-in-time liquidity. The remaining legitimate uses either extend this use case or require an external incentive. The contract is not a general yield optimizer or a complete allocation policy.

Permission model: Let S be the set of source adapter-market pairs that have both an active adapter and `canDeallocate == true`. Let D be the set of destination adapter-market pairs that have an active adapter and public absolute-cap headroom. Vault V2 caps also apply to every destination allocation.

The contract permits the following market-to-market paths:

$$S \times D.$$

The configuration does not store a source-destination pair. For example, if $S = \{A_1, A_2\}$ and $D = \{B_1, B_2\}$, the configuration permits all four combinations. It cannot permit only $A_1 \to B_1$ and $A_2 \to B_2$.

`canAllocateFromIdle` is one vault-wide flag. When it is true, idle assets can flow to every member of D. There is no separate idle permission for each destination.

The public absolute cap limits the final recorded allocation in a destination. It is not a maximum inflow for one call or one time interval. A Boolean `canDeallocate` permission does not enforce a source floor, `maxOut`, cooldown, or rate limit. Vault V2 caps bound accepted exposure. They do not establish that a route is timely, profitable, or beneficial to lenders.

Native penalty: Each public call must pay exactly the vault's `nativePenalty`. The payment is fixed per call and does not depend on `assets`. Therefore, a caller can reduce the fee per unit by moving more assets in one call.

The contract accrues the native penalty for the vault. Any current vault allocator can claim the full accrued amount and choose the receiver. The payment does not automatically increase vault assets or compensate vault shareholders.

This penalty is separate from the asset-proportional penalty charged by `VaultV2.forceDeallocate`. The two mechanisms have different payers, beneficiaries, and purposes.

Primary use cases: 1. Just-in-time borrow liquidity from another market

Assume a borrower wants to borrow q assets from Market B. Market B does not have q assets of available liquidity. The vault has at least q assets of withdrawable supply in an enabled Market A.

An external bundler can execute these actions atomically:

1. Call `reallocate` to move q assets from Market A to Market B.
2. Borrow q assets from Market B.

If the borrow fails, the complete bundle can revert. After a successful full borrow, Market B's available cash is close to its initial amount. Its total supply and total borrow have both increased by approximately q. The vault now funds credit exposure in Market B instead of Market A.

The borrower has a direct reason to pay the penalty: the reallocation makes an otherwise unavailable loan possible. This is the canonical Public Allocator use case described in the Morpho Public Allocator concepts.

2. Just-in-time borrow liquidity from idle assets

If `canAllocateFromIdle` is true, a borrower can use the same construction with `allocateFromIdle`. The vault can retain idle assets until a borrower needs them. The borrower then funds Market B immediately before the borrow.

This path avoids a withdrawal from Market A. It also consumes assets that were immediately available for vault withdrawals. The configuration must therefore treat the entire idle balance as publicly deployable, subject to the destination caps.

3. Debt refinancing

A borrower can use public allocation to enter a target market that does not have enough liquidity. An atomic bundle can:

1. Move vault liquidity to Market B;
2. Borrow from Market B;
3. Repay debt in Market C; and.
4. Complete the required collateral movement.

This can move debt to a market with a different rate, oracle, LLTV, or risk profile. The public allocator only supplies the target liquidity. The surrounding refinance logic must perform and validate the other actions. Morpho's SDK documentation describes refinance bundles that use shared Public Allocator liquidity.

4. Aggregation of fragmented liquidity across vaults

A large borrower can call the public allocator for several vaults before one borrow. Each vault contributes liquidity from its own approved envelope. This combines small liquidity fragments in Market B without merging the isolated markets or the vaults.

Each vault call has its own permissions, caps, available source liquidity, and native penalty. The current `reallocate` function accepts one source market per call. Moving assets from several sources therefore requires several calls.

5. Position increase in an otherwise illiquid market

An existing borrower may want to increase debt without changing markets. The same just-in-time flow can supply the missing liquidity before the additional borrow. This is economically identical to a new borrow, but it supports position management instead of initial market entry.

Secondary legitimate use cases: 6. Existing-borrower rate relief

Supplying assets to Market B reduces its utilization until borrowers consume the new liquidity. A large existing borrower can pay the penalty to reduce the borrow rate and future debt growth.

This action has no direct transfer to the caller. Its value comes from the caller's separate borrow position. It can reduce vault yield in Market B and can raise utilization in a source Market A. It is therefore a permitted externality, not an objective enforced by the contract.

7. Sponsored market liquidity

A market sponsor, protocol, DAO, or front end can pay for allocation into Market B. The sponsor may value deeper borrow liquidity, lower utilization, or greater activity in that market. Supply or borrow reward campaigns can provide another external reason to fund the call.

The contract does not verify the sponsor's objective. It does not distribute a keeper reward or prove that the vault receives more net yield. This use case requires an incentive outside `BluePublicAllocator`.

8. Pre-authorized market migration

An allocator can enable an old market as a source and give a replacement market destination headroom. Public callers can then move liquid assets from the old market to the replacement market without holding an allocator key.

This can reduce dependence on allocator availability. The present permissions remain active before and after the desired migration time. The contract does not verify an upgrade event, deadline, market state, or completion target.

9. Adapter migration

`reallocate` accepts different source and destination adapters. A vault can use this path to move a Morpho position between approved adapter deployments or adapter versions, if both adapters support the expected identifier scheme.

This is an operational use. It usually needs an external keeper or sponsor because the caller receives no direct value. The contract comments also require active adapters to be the expected Morpho adapter type. A different adapter can break the public cap calculation.

10. Permissionless failover or derisking

A vault can pre-authorize withdrawals from Market A and cap allocation into a safer or more liquid Market B. During an incident, any caller can execute the move even if the allocator is unavailable.

This is credible only when the allowed graph is narrow. The contract has no oracle or condition that proves Market A is unsafe or Market B is safer. Any caller can execute the move before an incident. `reallocate` also has no single vault-wide pause flag. An allocator must disable every relevant source or adapter to stop all such calls.

11. Externally funded allocation automation

A curator can use an offchain service to monitor utilization, rates, liquidity, or risk. The service can call `BluePublicAllocator` without receiving the full allocator role. This reduces key authority for the service.

The safety boundary remains the onchain permission graph and caps. The service's decision rule is not enforced by `BluePublicAllocator`. If several destinations have headroom, any other caller can choose a different destination first.

12. Large-shareholder self-help

A vault shareholder can move assets toward a destination that the shareholder expects to improve vault performance. The shareholder captures only its share of any benefit but pays the full gas cost and native penalty.

This can be rational for a sufficiently large shareholder. It is not a general keeper incentive. A caller with no vault shares does not receive the improved vault yield.

Economically rational adverse uses: The following actions can be rational for a public caller, although they are not beneficial allocator services.

13. Rate manipulation through external positions

A caller can hold a supplier position in source Market A and a borrow position in destination Market B. Moving vault assets from A to B can increase utilization and supplier yield in A while reducing utilization and borrow cost in B. The benefit can scale with the moved amount, the external positions, and time. The native penalty remains fixed per call.

14. Immediate borrowing after forced destination exposure

A borrower can fill all available destination headroom and immediately borrow the supplied liquidity. The destination cap authorizes the resulting vault exposure but does not bound the caller's loan size below that headroom. This is the intended shared-liquidity flow, but it can be adverse when the curator expected the cap to be used gradually.

15. Exit before pending bad debt

If Market B has known but unrealized bad debt, a current supplier can allocate vault assets into B and use the new cash to withdraw its own supply. The later loss then applies to the vault and the remaining suppliers. The public call does not create the bad debt. It can change who holds supply shares when the loss is realized.

16. Idle-liquidity depletion

`allocateFromIdle` can consume the vault's available idle balance. This can reduce immediately withdrawable vault liquidity. The destination caps limit exposure but do not preserve a minimum idle reserve.

17. Cap and share-accounting front-running

A caller can front-run another public allocation and fill the destination cap. The later call then reverts. A caller can also change source-market shares so a later exact-asset withdrawal no longer has sufficient shares. The implementation comments identify both cases.

18. Relative-cap manipulation with temporary capital

A caller can use a short-term vault deposit to change relative-cap headroom, then execute a public allocation. This requires capital. The implementation expressly identifies this behavior.

19. Repeated or reverse reallocation

If both markets are enabled sources and both have destination headroom, callers can move assets in either direction. The contract does not enforce a target portfolio or monotonic migration. A caller that benefits through external positions may pay the fixed penalty to reverse an earlier move.

Unsupported or weak interpretations: `BluePublicAllocator` does not natively provide the following functions:

- **General yield optimization.** It does not compare source and destination yield. It does not reward an unaffiliated caller for improving vault yield.
- **General risk optimization.** It does not compare oracles, collateral, LLTVs, bad debt, liquidity, or market health.
- **Pairwise route control.** Independent source and destination permissions produce $S \times D$ rather than selected edges.
- **Stable portfolio weights.** It does not store target allocations, source floors, or rebalance bands.
- **Market-to-idle withdrawal support.** `reallocate` must allocate the withdrawn assets into another adapter. `forceDeallocate` is the separate market-to-idle mechanism.
- **Guaranteed source liquidity.** A Morpho withdrawal still needs available market liquidity. Public allocation cannot unlock an illiquid source market.
- **Cross-asset routing.** The Morpho adapter requires each market's loan token to equal the vault asset. The public allocator performs no swap.
- **Arbitrary adapter support.** Its cap identifier assumes the expected Morpho adapter implementation.
- **One-call aggregation from many sources.** Each `reallocate` names one source and one destination. An external bundle must compose several calls and pay each required penalty.
- **Proportional compensation.** The native penalty does not scale with the moved amount, rate effect, liquidity effect, or realized loss.
- **Automatic lender compensation.** An allocator chooses the penalty receiver. The payment does not automatically accrue to the vault.

- **Incident pause.** Idle allocation has a vault-wide Boolean flag, but market-to-market reallocation has no equivalent global switch.

Recommendation: Treat `BluePublicAllocator` as a borrower-facing shared-liquidity primitive. Do not treat it as a complete public allocation policy.

For the current implementation:

- Treat every source permission as authority to withdraw the full liquid allocation;
- Treat every destination's headroom as exposure that any caller can fill at any time;
- Evaluate permissions as the full product $S \times D$;
- Keep `canAllocateFromIdle` false unless every destination can safely receive idle assets at adversarial timing;
- Do not enable idle or source assets that must remain immediately available;
- Limit aggregate destination headroom to the amount the vault accepts as publicly movable;
- Bundle the reallocation atomically with the borrow or refinance that gives the caller a legitimate incentive;
- Monitor caps, source liquidity, destination utilization, and native-penalty claims; and.
- Document that the fixed penalty is a call charge and does not price the amount moved.

If the contract is expected to support broader public portfolio management, add explicit source-destination routes, source `maxOut` limits, destination `maxIn` limits, minimum reserves, rate limits, and a vault-wide pause. Consider an amount-based fee and a fixed governance-defined beneficiary. Onchain conditions should enforce any required yield, utilization, or risk policy.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.2.6 Disallow same-route reallocation

Severity: Informational.

Context: BluePublicAllocator.sol#L98-L103.

Description: Unlike the public allocator for Vault V1.x vaults, the `BluePublicAllocator` allows de-allocating and then allocating into:

- The same market or.
- The same route (adapter, market) combo.

Recommendation: It might make sense to prevent the costly but no-op operation of moving assets from and then back to the same route.

Morpho: We acknowledge this issue. The public allocator reallocation algorithm (the one providing the list of reallocations to do batched with a borrow) should take that into account (reallocating into the same market is useless + could be costly).

5.2.7 Public Allocator comparison across MetaMorpho V1, V1.1, and Vault V2

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: `BluePublicAllocator` is not a direct port of the MetaMorpho `PublicAllocator`. The MetaMorpho contract controls public movement with directional flow budgets. The Vault V2 contract uses a source allowlist and a destination position ceiling.

The Vault V2 design adds adapter support, cross-adapter movement, idle allocation, and Vault V2's native absolute and relative caps. It also gives a public caller broader, reusable authority over each enabled source. The configuration must account for that difference.

Implementation summary:

Property	MetaMorpho V1/V1.1 `PublicAllocator`	Vault V2 `BluePublicAllocator`
Integration	Calls Morpho Blue and `MetaMorpho.reallocate` directly.	Calls `VaultV2.deallocate` and `VaultV2.allocate` through adapters.
Deployment scope	One immutable `MORPHO` address per deployment.	No protocol immutable; one deployment can store settings for many Vault V2 vaults.
Configuration authority	The vault owner or one PublicAllocator-specific `admin`.	Any current Vault V2 allocator.
Public operation	Many source markets to one destination market.	One source to one destination, including different adapters.
Source restriction	A quantitative `maxOut` budget.	Boolean `canDeallocate`; no amount or rate limit.
Destination restriction	A quantitative `maxIn` budget.	The final recorded allocation must not exceed the public `absoluteCap`.
Idle allocation	Not supported by `reallocateTo`.	`allocateFromIdle`, controlled by one vault-wide Boolean.
Vault controls	One MetaMorpho supply cap for each market.	Absolute and relative caps for every ID returned by the adapter.
Market identity	One Morpho market ID for the vault's direct position.	A per-adapter, per-market Vault V2 ID.
Adapter trust	Hard-coded Morpho integration.	Active adapters must follow `MorphoMarketV1AdapterV2` ID and accounting rules.
Native payment	One exact ETH fee for one multi-source reallocation.	One exact native penalty for each source-destination or idle-destination call.
Batching	Multiple withdrawals are included in `reallocateTo`.	Each `reallocate` names one source; the local `multicall` is nonpayable.

MetaMorpho directional flow budgets: For each market M, the MetaMorpho Public Allocator stores `maxIn[M]` and `maxOut[M]`. Let a public caller move x assets from market A to market B. The call applies these changes:

$$\begin{aligned} \text{maxIn}'[A] &= \text{maxIn}[A] + x, \quad & \text{maxOut}'[A] &= \text{maxOut}[A] - x, \\ \text{maxIn}'[B] &= \text{maxIn}[B] - x, \quad & \text{maxOut}'[B] &= \text{maxOut}[B] + x. \end{aligned}$$

Thus, the following quantity is invariant for each market changed by the public allocator:

$$\text{maxIn}[M] + \text{maxOut}[M].$$

The move consumes the source's public outflow budget and the destination's public inflow budget. It creates the same amount of capacity for the reverse move. A caller cannot remove more than `maxOut` from an enabled source, even when several destinations have room.

The contract limits each settable flow cap to `type(uint128).max / 2`. This leaves enough space for later cap transfers without overflowing the `uint128` fields. The public function also requires:

- At least one withdrawal;
- A nonzero amount for each withdrawal;
- Enabled source and destination markets;
- Sorted, unique source market IDs;
- No source market equal to the destination market;
- Enough vault supply for every withdrawal;
- Enough `maxOut` at every source; and.
- Enough `maxIn` at the destination.

The contract then calls `MetaMorpho.reallocate` once. The transaction reverts atomically if any check or Morpho operation fails.

Vault V2 source permission and destination ceiling: For a source adapter-market pair S and destination adapter-market pair D, `BluePublicAllocator.reallocate` requires:

$$\begin{aligned} \text{active[adapter}(S)\text{]} &= \text{true}, \\ \text{active[adapter}(D)\text{]} &= \text{true}, \\ \text{canDeallocate[}S\text{]} &= \text{true}, \\ \text{allocation[}D\text{]} &\leq \text{publicAbsoluteCap[}D\text{]}. \end{aligned}$$

The first three conditions are checked before the move. The last condition is checked after the destination allocation. The transaction remains atomic: if the allocation or final check fails, the earlier deallocation also reverts.

> ⚠ **Warning:**
> Unlike `maxOut`, `canDeallocate` is not consumed. One successful call does not reduce the authority available to the next caller. A caller can remove the full liquid source allocation over one or more calls. The caller only needs enough destination headroom and must pay the fixed native penalty for each call.

The destination cap has balance semantics. If the current destination allocation is a and its public cap is c, the available public headroom is at most:

$$\max(0, c - a).$$

This is not an inflow budget for one call or one time interval. Deallocation from the destination restores headroom because it lowers the recorded allocation. No Public Allocator cap value changes during the move.

The public `absoluteCap` is stored as `uint256`, and its setter does not impose a maximum. A value above every feasible Vault V2 allocation effectively disables this extra public ceiling. Vault V2's own `uint128` absolute caps and relative caps still apply.

Effective Vault V2 destination limits: `MorphoMarketV1AdapterV2.ids` returns three IDs:

1. An adapter-wide ID;
2. A collateral-token ID; and.
3. An adapter-market ID computed as `keccak256(abi.encode("this/marketParams", adapter, marketParams))`.

`VaultV2.allocate` applies its own absolute and relative caps to all three IDs. `BluePublicAllocator` then checks its separate absolute cap only on the third ID. The effective limit is therefore the strictest applicable limit among:

- The BluePublicAllocator adapter-market cap;
- The Vault V2 adapter-wide absolute and relative caps;
- The Vault V2 collateral absolute and relative caps; and.
- The Vault V2 adapter-market absolute and relative caps.

This model is more expressive than one MetaMorpho market supply cap. However, it controls accepted exposure. It does not limit public source outflow.

Authorization differences: The MetaMorpho Public Allocator stores one `admin` for each vault. The current admin and the vault owner can set the admin, fee, and flow caps. The vault owner always keeps this configuration authority.

`BluePublicAllocator` has no local admin. Every setter and native-penalty claim checks `IVaultV2(vault).isAllocator(msg.sender)`. The owner, curator, and sentinels do not pass this check unless they are also registered as allocators. Removing an account's allocator role removes its configuration access.

Both public allocator contracts must themselves have the allocator permission required by the target vault. Otherwise, their calls to the vault revert.

Native fee and penalty differences: Both contracts require an exact native payment and accrue it by vault. A failed allocation reverts the accrual.

The MetaMorpho implementation stores `fee` and `accruedFee` as `uint256`. The owner or local admin can transfer the full accrued fee. The transfer uses Solidity's `transfer` operation.

The Vault V2 implementation packs `nativePenalty` and `accruedNativePenalty` into `uint120`. It rejects a configured penalty above `type(uint120).max`. Any current Vault V2 allocator can claim the full amount to an arbitrary receiver. The transfer uses `call` and checks its result.

The cost also has different batching semantics. MetaMorpho can withdraw from many sources for one fee. Vault V2 charges one penalty for each source-destination call. The nonpayable `multicall` cannot fund calls with a nonzero penalty. *It can delegatecall zero-value public operations when the configured penalty is zero*.

Detailed security and configuration notes: 1. A source permission has no quantitative outflow limit

`canDeallocate == true` permits repeated withdrawals from that source. The native penalty is fixed per call and does not scale with `assets`. A caller can therefore move a large amount in one call, up to the `uint128` input and the available source liquidity.

This is the largest semantic difference from `maxOut`. It is not a cap bypass: the destination still has to pass the public and Vault V2 cap checks. It is a broader source-side permission.

If many destinations have headroom, a caller can divide the source allocation among them. The Public Allocator does not preserve a source floor or minimum withdrawal reserve.

2. The adapter invariant is not enforced by `BluePublicAllocator`

`setIsActiveAdapter` accepts any address from an authorized Vault V2 allocator. It does not verify the adapter factory, adapter code, parent vault, or ID scheme.

The public cap is correct only if the active adapter updates the exact adapter-market ID computed by `vaultBlueId`. A different adapter can make the final check read an unrelated allocation. It can also make the public call revert.

Vault V2 still requires the address to be a vault adapter when execution starts. An adapter registry can also restrict which adapters the curator may add. These controls can enforce the intended deployment policy, but `BluePublicAllocator` does not enforce the policy itself.

3. The public cap is per adapter, not global per Morpho market

The adapter address is part of the capped ID. Two adapters that access the same Morpho market have different BluePublicAllocator caps and different Vault V2 adapter-market IDs.

Their combined position is not subject to one shared market cap unless another shared Vault V2 ID provides that limit. The collateral-token ID can limit aggregate collateral exposure, but it does not identify one Morpho market.

MetaMorpho holds one direct vault position for each Morpho market. Its flow caps therefore use one global market ID within that vault.

4. Same-position reallocation is allowed

The MetaMorpho contract rejects a destination market that also appears in the withdrawal list. `BluePublicAllocator` does not require the source and destination IDs to differ.

A caller can deallocate from and allocate back to the same adapter-market position. `MorphoMarketV1AdapterV2` states that supply and withdrawal rounding losses are realizable. A same-position round trip therefore exposes these operations to public callers. Repeated calls can accumulate dust losses when a rounding difference occurs.

The economic effect can be small compared with gas and the native penalty. The behavior is still a wider public surface than the MetaMorpho implementation.

5. Zero-asset public operations are allowed

The MetaMorpho contract rejects every zero withdrawal. `BluePublicAllocator` does not require `assets > 0` in `reallocate` or `allocateFromIdle`.

A zero-asset call can still enter Vault V2 and adapter accounting. It can accrue interest or refresh a recorded allocation. This can be useful as a permissionless accounting update, but the interface and comments do not state that purpose.

6. The BluePublicAllocator cap is not a vault-wide invariant

The cap is checked only after calls made through `BluePublicAllocator`. Allocations made directly by another vault allocator do not use this cap. Deposits routed through the liquidity adapter also do not use this cap.

These other paths can leave the recorded allocation above the public cap. In that state, a later public allocation to the position reverts until the allocation returns below the cap or an allocator raises the cap.

Vault V2's own caps continue to apply to every allocation path. Interest and donations can still make a recorded or economic position exceed a cap after an accepted allocation.

7. Relative-cap manipulation is new to this comparison

MetaMorpho's relevant market cap is absolute. Vault V2 also has relative caps. `firstTotalAssets` prevents a simple same-transaction flash-loan bypass, but it does not prevent a caller from using capital for a short-term deposit.

A public caller can deposit before a public allocation and increase the base used by a relative-cap check. The caller can withdraw later. The remaining allocation can then be above the same relative percentage of the reduced vault assets. The implementation comments identify this behavior and state that it requires capital.

8. Public calls can be front-run

Another permitted public call can fill destination headroom before a pending allocation. The pending call then reverts at a Vault V2 or BluePublicAllocator cap check.

A source-market action can also change shares or available liquidity before a pending exact-asset deallocation. The later deallocation can then revert. The implementation comments identify both cases.

MetaMorpho flow caps can also be consumed by an earlier public call. The main V2 difference is that final position caps and adapter share accounting create additional failure conditions.

9. Market validation moved into the vault and adapter

The MetaMorpho Public Allocator checks that every source and destination market is enabled. It also accrues Morpho interest and checks the vault's expected supply before it calls `MetaMorpho.reallocate`.

`BluePublicAllocator` does not validate a market when an allocator stores its settings. At execution, Vault V2 checks the adapter, and `MorphoMarketV1AdapterV2` checks the loan token, IRM, position shares, and Morpho operation. The Vault V2 allocation path then enforces its caps.

This permits settings to be prepared before a market is usable. It also allows stale settings to remain after adapter or vault configuration changes.

10. The operation shapes differ

MetaMorpho accepts a sorted list of sources and one destination. One call can aggregate several withdrawals and pays one fee.

Vault V2 accepts one source and one destination. Moving funds from several sources requires several calls or an external atomic bundle. Each successful call pays its own configured penalty.

Vault V2 also supports cross-adapter moves and public allocation from idle. The MetaMorpho Public Allocator has neither function.

MetaMorpho V1 and V1.1 compatibility: The relevant `owner`, `config`, and `reallocate` selectors and data layouts are compatible with both MetaMorpho versions.

MetaMorpho V1 checks `config[id].enabled` in its withdrawal branch. Its supply branch uses a nonzero supply cap as authorization.

MetaMorpho V1.1 checks `config[id].enabled` for every allocation entry before it selects the withdrawal or supply branch.

The Public Allocator checks every source and the destination for `enabled` before it calls either version. Its public path therefore applies the same enabled-market restriction to both versions.

Verification: The focused `BluePublicAllocatorTest` suite passed at Vault V2 commit `4c7c110a9a3c3ce1ec545fff3b8a832f16cedfcc`: 31 tests passed, 0 failed, and 0 were skipped. The suite covers configuration authority, movement within and across adapters, idle allocation, public and vault cap failures, active-adapter checks, native-penalty accounting, and penalty claims.

The existing suite does not state a policy for zero-asset calls or same-position reallocation. Passing tests therefore confirm the tested implementation behavior. They do not resolve whether those two cases are intended.

Recommendation: Treat the two cap models as different policies. Do not translate a MetaMorpho `maxOut` value into `canDeallocate == true` without a separate decision to permit full public withdrawal from that source.

For the current Vault V2 design:

- Treat `canDeallocate == true` as reusable authority over the source's full liquid allocation;
- Treat every destination's public headroom as exposure that any caller can fill at any time;
- Bound the combined exposure of duplicate adapters when they can reach the same Morpho market;
- Enforce the expected adapter factory, parent vault, and identifier scheme in deployment policy or in the contract;
- Keep Vault V2 adapter-wide, collateral, and adapter-market caps consistent with the public cap;

- Account for direct allocator actions and liquidity-adapter deposits, which do not use the public cap;
- Account for capital-backed relative-cap manipulation and public-call front-running; and.
- Set the native penalty as a call charge, not as an amount-based outflow limit.

If the intended policy needs MetaMorpho-like flow control, add quantitative source `maxOut` and destination `maxIn` budgets. Update them atomically after each move.

Unless same-position refreshes are required, reject `deallocateId == allocateId`. Unless zero-asset accounting updates are an intended public function, require `assets > 0`. Add tests that fix the selected policy for both cases.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.2.8 Ambiguous naming in allocateFromIdle and setCanAllocateFromIdle

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: The `allocateFromIdle` path uses deallocation-oriented names that don't match what the function does.

- `setCanAllocateFromIdle` sets `canAllocateFromIdle` but names its parameter `newCanDeallocate`.
- `allocateFromIdle` checks `canAllocateFromIdle` but reverts with `CannotDeallocate()`.

The behavior is correct, but the naming is ambiguous.

Recommendation: Rename these identifiers such that each capital source is unambiguous.

Morpho: Fixed in PR 958.

Spearbit: PR 958 adopts ”pull” naming throughout: the `setCanAllocateFromIdle` parameter is renamed to `newCanPullFromIdle`, the storage flag becomes `canPullFromIdle`, and the revert identifiers become `CannotPullFromIdle()` and `CannotPullFromMarket()`.

5.2.9 Inaccurate multicall comment about reallocate & allocateFromIdle

Severity: Informational.

Context: BluePublicAllocator.sol#L34.

Description: The comment on `multicall` states:

```
/// @dev Nonpayable so it cannot be used with reallocate and allocateFromIdle.
```

`multicall` is nonpayable, so `msg.value` inside the delegatecalls is always 0, which is why it can't carry a native penalty.

But that only blocks `reallocate` and `allocateFromIdle` when the penalty is nonzero. If a vault's `nativePenalty` is 0, the `msg.value == nativePenalty` check passes with 0, so both functions can be batched through `multicall` after all.

The comment presents an absolute rule where there's actually an exception.

Recommendation: Reword the comment to note the zero-penalty exception.

Morpho: The `nativePenalty` was removed. This is no longer the case with a penalty in loan asset so the comment was removed in the PR 966.

5.2.10 Additional configuration risks introduced by `BluePublicAllocator`

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: Whenever a curator makes a configuration change in a `VaultV2` which used to or currently uses `BluePublicAllocator`, they should always check the configuration of `BluePublicAllocator` as well.

Otherwise it's very easy to miss an adapter that may be allocated to by the public, for example:

- Assume `VaultV2` uses `BluePublicAllocator`.
- Curator removes it as an allocator from the vault. This does not clear the `isActiveAdapter` mapping.
- After a long period of time, `BluePublicAllocator` is added back as an allocator.

It is easy to assume that `BluePublicAllocator` will start from a fresh state (ie. have no active adapters) when re-added, but this is not true.

The same pattern exists if an active adapter is removed from the vault, then added back later on.

Recommendation: A possible fix would be to introduce a nonce which can be incremented to clear the vault's entire state in `BluePublicAllocator`:

```
mapping(address vault => mapping(uint256 nonce => mapping(address adapter => bool))) public
↪  isActiveAdapter;
```

Alternatively, it is sufficient to simply document this for curators.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.2.11 Additional risk introduced to `VaultV2` with public allocations

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: Allowing public allocations introduces additional risks to `VaultV2`, which was previously restricted to only trusted allocators.

(1) Allocations/deallocations in `VaultV2` can be forced to revert via front-running. For example:

- `VaultV2` has X assets in a market.
- Sentinel calls `VaultV2.deallocate()` with `assets = Y`.
- Attacker front-runs and calls `BluePublicAllocator.reallocate()` with `assets = X - Y + 1`.
- Sentinel's deallocation reverts due to insufficient assets in the market.

This seems generally acceptable considering there is a `nativePenalty`, and `allocate()` / `deallocate()` can just be called again with an adjusted `assets` amount.

The only case where this could be an issue is if `deallocate()` is called by the sentinel in some emergency scenario, and the sentinel happens to be a timelocked role (eg. sentinel is a safe). In this case, the calldata for

`deallocate()` with the new `assets` amount has to go through the entire timelock again, so there may be a more significant impact.

(2) When `burnShares()` is called for a market in an active adapter, it is problematic if someone can allocate into that market to increase `supplyShares` afterwards, since `burnShares()` (which is timelocked) would need to be called again.

To avoid this:

- The curator/allocators must ensure the market cannot be publicly allocated into before calling `burnShares()`.
- Or `burnShares()` must be batched together in a transaction with the subsequent `deallocate()` with `assets = 0` call to `VaultV2`.

Recommendation: Consider documenting the risks above, and how they can be avoided.

Morpho: Acknowledged.

(1) The sentinel can always lower the caps to zero in case of emergency, blocking any allocation attempts from the PA (and other allocators).

(2) It is documented here.